

Automated Verification for Health Economic Models in R: A Hands-On Tutorial

Nathan Green, *UCL*
Daniel Gallardo, *ICON Clinical Research*

July 13, 2026

Abstract

Health economic and decision-analytic models have become increasingly complex, and there has been a shift toward open-source programming languages like R. While best-practice guidelines consistently emphasize the conceptual importance of model verification and validation, there is a lack of practical, hands-on guidance showing *how* to implement automated software testing techniques within health economic modelling workflows. This tutorial provides a step-by-step guide to integrating automated verification using unit testing into health economic modelling in R and the `testthat` package for more trustworthy and robust code.

1 Introduction

Health economic modelling has undergone a rapid methodological paradigm shift in recent years. For decades, decision-analytic modelling relied almost exclusively on spreadsheet software such as Microsoft (MS) Excel.[7] However, as health economic models have grown in structural complexity and computational demand, driven in part by increasing microsimulation and rigorous Probabilistic Sensitivity Analysis (PSA), the field has increasingly migrated toward open-source programming languages such as R. This transition offers undeniable benefits to the health economics community, including superior computational efficiency, advanced statistical capabilities, and the ability to seamlessly integrate data synthesis, model execution, and post-processing with advanced visualisations. [10]. Some tutorials provide step-by-step guidance to help with this transition [6, 14, 16].

The health economics community has long recognized the importance of model verification and validation. Prominent frameworks, including those developed by the International Society for Pharmacoeconomics and Outcomes Research and the Society for Medical Decision Making (ISPOR-SMDM) Modelling Good Research Practices Task Force [15, 5, 3], consistently emphasize the need to rigorously model check logic and internal consistency. Zorginstituut Nederland (ZIN) is currently the most explicit regulatory body globally regarding R-based submissions. They published a guideline for building cost-effectiveness models in R which mandates strict standardization to ensure models are transparent, fully executable, and adaptable. [22] Furthermore, recent literature has provided excellent conceptual overviews on transparency and reproducibility [1]. However, much of the existing guidance focuses on high-level conceptual frameworks or late-stage validation techniques, such as cross-model comparisons or validating final outputs against empirical data [17]. There remains a significant gap in the literature regarding practical, hands-on guidance demonstrating exactly how modellers can verify the underlying mechanics of their code during the development phase.

To address this gap, health economists must look beyond traditional Health Technology Assessment (HTA) practices and adopt foundational quality-assurance techniques from the field of software engineering specifically. It is common for the health economics modeller to test that their model behaves as they expect but only do it informally, in an ad-hoc way. Software engineers have long automated how they test their code. This improves the workflow by detecting and producing fewer bugs, better code structure, and robust code [21].

The objective of this tutorial is to demystify automated testing and demonstrate its practical implementation within health economic modelling. Using a realistic Markov cohort model built in R, this paper provides a didactic, step-by-step guide to writing, executing, and interpreting automated tests using the widely adopted *testthat* package [20]. By integrating the framework presented in this tutorial, modellers can establish automated checks that proactively identify and resolve code-level errors before they can influence reimbursement decisions.

2 Hierarchy of Testing

Testing can be performed at different levels of abstraction. Figure 1 illustrates the adapted software engineering “test pyramid” framework applied to health economic mod-

elling. The pyramid establishes a structural hierarchy where a high volume of granular, low-level tests forms the foundation, supporting a smaller number of complex, high-level integration and system tests. Moving upward through the pyramid reflects an increase in the scope and computational cost of the evaluation, alongside a corresponding decrease in execution speed.

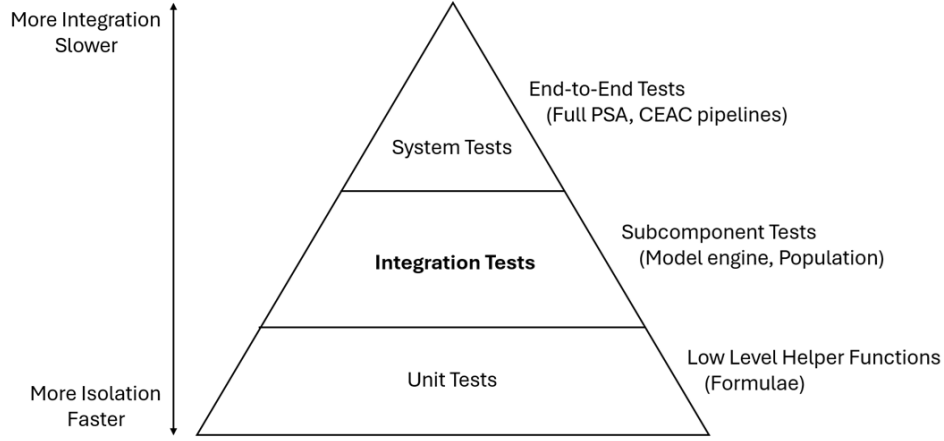


Figure 1: Software engineering “Test Pyramid” for health economic modelling.

2.1 Level 1: Unit Testing (Lowest Level)

In software development, unit testing is the practice of isolating the smallest testable parts of an application referred to as “units”, and independently verifying that they function correctly. In the context of a health economic model built in R, a “unit” is typically a single, self-contained function. For example, rather than waiting until the entire Markov trace or Probabilistic Sensitivity Analysis (PSA) is completely generated to check the final Incremental Cost-Effectiveness Ratio (ICER) for face validity, unit testing evaluates the model’s fundamental building blocks. This is shown in the base of Figure 1.

2.2 Level 2: Integration Testing (Mid Level)

This is defined as verifying that multiple components (input structures, simulation loops, and summary extractors) interact correctly to produce logically sound results. An HTA example is when executing a modeller runner function, e.g. `run_model()`, with specific parameters and asserting that the output behave as expected. The tests are designed to catch silent logic bugs, index shifts, and broken dependencies between clinical inputs and economic outputs. Within integration testing there are more or less isolated tests which may be called subsystem or application tests. This is shown in the middle layer of Figure 1. To understand the level of rigor required for this testing, health economists can look to other safety-critical fields. For example, integration testing in the automotive industry utilizes highly regulated compliance frameworks such as Automotive SPICE (A-SPICE) and the Federal Motor Vehicle Safety Standards (FMVSS). Managed by the National Highway Traffic Safety Administration (NHTSA), these strict frameworks dictate how integration testing must be designed, executed, and documented to govern the safety performance requirements for motor vehicles. [19, 13]

2.3 Level 3: System or End-to-End Testing (High Level)

System, or end-to-end testing can be defined as executing the entire decision-analytic pipeline from raw data ingestion to final policy outputs. In an HTA context, an example would be running a 10,000 simulation Probabilistic Sensitivity Analysis (PSA) to verify that the pipeline executes without memory leaks, handles missing data gracefully, and successfully outputs an Incremental Cost-Effectiveness Ratio (ICER). For example, a unit could be a function that calculates age-dependent background mortality, a function that applies half-cycle corrections and discounting to quality-adjusted life-years (QALYs), or a function that populates a transition probability matrix for a given simulation cycle. By verifying each of the components in isolation, modellers can ensure foundational accuracy and pinpoint exactly where a logical or mathematical error originates before it propagates through the rest of the model. This is shown in the top of Figure 1.

3 Testing and Health Economics Modelling

While foundational software engineering literature emphasizes the isolation of pure functions via unit testing, health economic models frequently require a broader approach due to the absence of a simple ‘test oracle’ (an absolute known output for complex multi-state traces). Consequently, the automated checks presented in this tutorial primarily operate as integration tests. Rather than verifying isolated lines of code, these tests confirm that the integration of input parameters, transition matrices, and simulation engines preserves foundational economic and clinical logic (e.g., verifying the relation that future value decreases as discount rates rise). Ultimately, this automated suite serves as a rigorous engine for internal model verification—ensuring the computational pipeline executes exactly as intended—which is a necessary prerequisite to external clinical validation [4, 18].

4 The *testthat* Package

It is technically possible to write ad-hoc tests using base R commands (such as `stopifnot()`), the *testthat* package has emerged as the definitive standard for automated testing within the R ecosystem [20]. To setup a package to use *testthat*

```
usethis::use_testthat(3)
```

Note that the `::` syntax is used to be explicit about the *usethis* package to which the function belongs. A folder `tests/testthat` will be created, adds it to the `DESCRIPTION` file and creates a file `tests/testthat.R` that runs all the tests when R checks are run. As functions are defined in a package, in the files below `R/`, the corresponding tests are added to `.R` files in `tests/testthat/`. Filenames with tests in must start with `test`. The *usethis* package has some convenience functions, including `use_test(name)` which creates and opens a test file called `tests/testthat/test-name.R`.

For health economists, *testthat* provides a structured, highly readable syntax built around plain-English naming conventions. This reduces the barrier to entry, making test scripts almost as readable as the conceptual methodologies they are verifying. These are called *expectations* and are the most basic unit of using *testthat*. All expectations start with `expect_` (e.g., `expect_equal()`, `expect_error()`, `expect_type()`), have the actual and expected results as arguments, and they throw an error if these two things

don't agree. Collections of related expectations are grouped together in to what *testthat* calls tests. A group is wrapped by the `test_that("<description>", ...)`. A file then hold multiple related tests. Furthermore, *testthat* allows modellers to bundle dozens or hundreds of individual unit tests into a cohesive “test suite.” This suite can be executed seamlessly with a single command, providing an immediate, visual summary of which model components passed and which require debugging. For an in depth reference of the *testthat* package see [21].

5 Health Economics Modelling Test Suite

To ensure the foundational accuracy of the underlying computation and prevent silent error propagation, modellers can implement a standardized suite of automated unit tests. Table 1 presents an example of practical verification tests, extreme-value checks, and logical assertions adapted from established health technology assessment (HTA) quality assurance frameworks. Each test is assigned a unique identifier and belongs to a set of tests, e.g. related to cost or QALY calculations, or population counts. For each test, a test condition is defined which specifies the scenario input parameter of interest, e.g. setting the discount rate to 0. Given this, then there is an expected result from running the model, e.g. with no discounting then the total QALYs are the same as the life expectancy.

Specifically, these test scenarios operationalize the internal consistency and extreme-value checks introduced by McCabe and Dixon (2000) [12], the comprehensive “black-box” verification criteria formalized by Tappenden and Chilcott (2014) [18], the automated decision-analytic checking principles outlined by Dasbach and Elbasha (2017) [4], and the technical validation standards reinforced by the CHEERS statement (Husereau et al., 2013) [9]. The following section will practically demonstrate how to translate these conceptual checks into (automated) *testthat* expectations.

6 Case Study: A Discrete-Time Markov Model

6.1 Model Overview

A 3-state Sick-Sicker-Dead-type cohort model is considered. Figure 2 gives a schematic representation of the cohort Markov model.

Define the baseline input values for the 3-state model with two intervention scenarios of ‘Without Drug’ and ‘With Drug’, indexed by $s = 1$ and 2, respectively. The state occupancy costs and utilities are

$$C^1 = \begin{pmatrix} 100 \\ 200 \\ 0 \end{pmatrix}, \quad C^2 = \begin{pmatrix} 500 \\ 200 \\ 0 \end{pmatrix}, \quad Q^s = \begin{pmatrix} 0.9 \\ 0.6 \\ 0 \end{pmatrix},$$

and the transition probabilities are

$$P^1 = \begin{pmatrix} 0.75 & 0.2 & 0.05 \\ 0 & 0.9 & 0.1 \\ 0 & 0 & 1 \end{pmatrix}, \quad P^2 = \begin{pmatrix} 0.85 & 0.1 & 0.05 \\ 0 & 0.95 & 0.05 \\ 0 & 0 & 1 \end{pmatrix}$$

ID	Group	Test Condition	Expected Result
T01	QALY	Set discount rate = 1	All future QALYs = 0
T02	QALY	Set discount rate = 0 (assuming utility = 1)	Total QALYs equal Life Expectancy (LE)
T03	Population	Set relative treatment effects (odds ratio, hazard ratio, risk ratio) = 1	All treatments produce equal estimates of total LYGs and total QALYs
T04	Population	Set mortality transition probabilities = 0	LE = time horizon
T05	Population	Set mortality transition probabilities = 1	LE = 1
T06	QALY	Set all living health state utility parameters = 1	QALY gains exactly equal LYGs
T07	QALY	Apply standard positive discount rate vs. 0 rate	Discounted QALYs < undiscounted QALYs for all treatments
T08	QALY	Set QALY discount rate = ∞	QALY gains after time 0 tend towards zero
T09	Cost	Set intervention costs = 0	ICER is reduced
T10	Cost	Increase intervention costs	ICER is increased
T11	Cost	Set cost discount rate = 0	Discounted costs = undiscounted costs for all treatments
T12	Cost	Set cost discount rate = ∞	Costs after time 0 tend towards zero
T13	Input	Sample parameters from probabilistic distributions	Range of sampled values does not violate distribution boundaries (e.g., beta samples lie in $0 \leq x < 1$, lognormal samples lie in $x > 0$)

Table 1: Example suite of validation tests for health economic models.

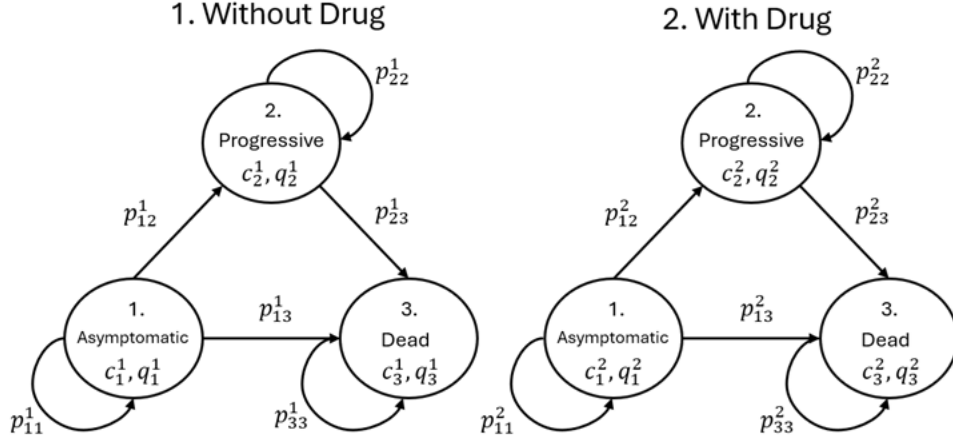


Figure 2: Markov model diagram for Without Drug and With Drug scenarios.

Define a cohort of size $n = 1000$ to all start in state one (Asymptomatic) at time $t = 0$. Simulate for 15 cycles.

6.2 Model Architecture

To effectively implement automated verification using the *testthat* package, the underlying R code cannot be written as a single monolithic script. Instead, the model’s architecture must embrace the software engineering principle of “separation of concerns,” dividing the computational pipeline into discrete, highly modular components. This modularity is what fundamentally makes a model “testable.”

For the cohort Markov model, the architecture is explicitly separated into three distinct domains. First, the *input parameters* (such as transition matrices, cost vectors, and utility weights) are isolated into independent lists. Second, the *simulation engine* (the `ce_markov()` function) accepts the input parameters, executes the matrix multiplication across the specified time horizon, and returns a single object with all outputs. Third, *helper functions* simplify the code with small, reusable functions that abstract away common or repetitive tasks in the whole modelling pipeline so that the main logic stays clean.

By compartmentalizing the code in this manner, modellers can isolate and test individual units of logic. For example, we can pass an extreme boundary value into the simulation engine and isolate the output extractor to verify that a specific ICER logic holds, without having to run an entire PSA pipeline. To facilitate the automated test suite detailed in Section 7, we define a standardized baseline input list, `base_params`, which serves as the universal reference point for all subsequent logic checks.

7 R Test Code Examples

In this section, we will step through how to use the *testthat* package in R to automate validation tests of a cost-effectiveness analysis using the defined cohort Markov model.

7.1 Input Parameters

Translating the information from 6.1 into R code input values

```

# basic test data
t_names <- c("without_drug", "with_drug") # treatments
s_names <- c("Asymptomatic", "Progressive", "Dead") # states

# base CE model input parameters
base_params <- list(
  # population at time 0 all in Asymptomatic state
  start_pop = c(1000, 0, 0),
  # transition probability matrix
  p_matrix = array(c(0.75,0.2,0.05,0,0.9,0.1,0,0,1,
                    0.85,0.1,0.05,0,0.95,0.05,0,0,1),
                  dim = c(3,3,2),
                  dimnames = list(s_names, s_names, t_names)),
  # state costs
  state_c_matrix = matrix(c(100, 200, 0, 500, 200, 0),
                          nrow = 2, byrow = TRUE),
  # state utilities
  state_q_matrix = matrix(c(0.9, 0.6, 0),
                          nrow = 2, ncol = 3),
  n_cycles = 15 # time horizon
)

```

7.2 Helper Functions

A convenient idea is to use *setters or mutators*, and *getters or accessors* to add a layer of abstraction to interact with the models. These are simple helper functions which change a value in an object, such as a list, or return the value of a part of an object. A setter can be used to modify part of the `base_params` list we have just created without redefining the whole thing. A getter can easily extract QALYs, costs, Life Expectancy, etc. from the results of a model run. For example, to set the discount rate, the following code can be used,

```

# setter helper function
set_discount_rate <- function(input, discount_rate) {
  input$discount_rate <- discount_rate
  input
}

```

and to return the total QALYs after a model run the following code.

```

# getter helper function
get_qalys <- function(res) {
  res$total_QALYs
}

```

The inner workings of the setters and getters are particular to a model but all the user needs to know is `set_discount_rate()` or `get_qalys()` and not the structure of the object that is passed to or returned from that model.

As an extension to this idea, to facilitate the following tests, define a *wrapper* function `run_model()`. This allows us to pass the default dataset but override specific parameters we want to test (like the discount rate). This can be thought of as a type of setter function as part of the call to `ce_markov()`.

```

# CE model convenience wrapper function
run_model <- function(input_data, ...) {
  # merge default input_data with manual overrides (...)
  updates <- list(...)

  # replace with new values
  args_list <- modifyList(input_data, updates)

  # call the main model function
  do.call(ce_markov, args_list)
}

```

Using ... (ellipses) allows us to write clean tests such as `run_model(base_data, discount_rate = 0)` without having to manually reconstruct the entire input list every time. Three types of validity checks will be written next.

7.3 Discounting Tests

A fundamental property of health economic models is that the present value (*PV*) of a future value (*FV*) decreases as the time t increases further into the future and the discounting increases. For example, the present value of a QALY at time t is given by the following.

$$q_{PV} = \frac{q_{FV}}{(1 + r)^t}$$

Testing the model code when there is discounting in the model is crucial, since it will modify the total costs and health values and the decisions. The validation tests that correspond to the effect of discounting in Table 1 are T1, T2 T3 (for QALYs), T5, T7 (for costs) and T8 for LE. To demonstrate test T7 in Table 1, where the test case focused on QALYS, can be broken down into three steps:

- A: Run model with discount rate $r = 0\%$, i.e. $q_{PV} = q_{FV}$ (undiscounted).
- B: Run model with discount rate $r = 3.5\%$ (discounted).
- Check expectation that q_{PV} for B $<$ q_{PV} for A.

Below, the code needed to perform unit tests to verify discounting logic is detailed. This Markov model is run with with 3.5% discounting and assign the results to `results_discounted`, and with 0% discounting and assign the results to `results_undiscounted`. This results in how to ‘get’ the QALYs from these and compare the scenarios with and without drugs in expectation statements.

```

test_that("Discounted QALYs are lower than undiscounted QALYs", {
  # run model with standard discounting (3.5%)
  results_discounted <-
    ce_markov(start_pop, p_matrix, state_c_matrix, trans_c_matrix,
              state_q_matrix, n_cycles, discount_rate = 0.035)

  # run model with NO discounting (0%)
  results_undiscounted <-
    ce_markov(start_pop, p_matrix, state_c_matrix, trans_c_matrix,

```

```

    state_q_matrix, n_cycles, discount_rate = 0.0)

# alternatively:
# results_discounted <-
#   run_model(base_params, discount_rate = 0.035)
# results_undiscounted <-
#   run_model(base_params, discount_rate = 0.00)

# extract the total QALYs using getter
# could use direct list access instead
qaly_discounted <- get_qalys(results_discounted)
qaly_undiscounted <- get_qalys(results_undiscounted)

# expectation: discounted value must be less than undiscounted value
expect_lt(qaly_discounted["without_drug"],
          qaly_undiscounted["without_drug"])
expect_lt(qaly_discounted["with_drug"],
          qaly_undiscounted["with_drug"])
})

```

Next, test T8 from Table 1 is implemented, namely that the edge case of a sufficiently large discounting rate r results in the costs and QALYs reducing to 0, i.e. $q_{PV} \rightarrow 0$ as $r \rightarrow \infty$. In this case, the `tolerance` argument is used in `expect_equal()` which allows the expectation to return TRUE if the expected and actual values are within some margin of error. In the remaining examples, the `run_model()` helper function is used.

```

test_that("High discounting results in zero totals", {
  res_high_disc <- run_model(base_params, discount_rate = 1000)

# cost
expect_lt(get_qalys(res_high_disc)[1], 1)
expect_equal(get_qalys(res_high_disc)[1], 0, tolerance = 0.01)

# QALYs
expect_lt(get_qalys(res_high_disc)[2], 1)
expect_equal(get_qalys(res_high_disc)[2], 0, tolerance = 0.01)
})

```

7.4 Unit Costs and Utilities Tests

Tests for unit costs and utilities are often boundary value type tests that assign zeros and ones, and are simple calculations that could often be performed without even running the model and can be thought of as “sanity checks.” Test T9 in Table 1 is implemented and has the condition that all costs in the ‘With Drug’ scenario are assigned a value of zero. The expectation is that the model must then output zero total cost. Further, the ICER for the case with zero costs for the intervention is necessarily reduced from the ICER using the base case costs. In the Markov model, the state unit cost vector for intervention scenario ‘With Drug’ is

$$C^2 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

The test case implemented in the code is to set `state_c_matrix` for the ‘With Drug’ scenario to 0. The expectation is that `run_model()` returns a total costs = 0.

```
test_that(
  "Zero unit cost inputs result in zero total cost outputs", {
    # create matrices of zeros with the correct dimensions
    zero_state_c <- base_params$state_c_matrix[2, ] * 0

    # run model with baseline costs
    res_base <- run_model(base_params)

    # run model with zero unit costs
    res_zeros <- run_model(base_params,
                          state_c_matrix = zero_state_c)

    res_zeros_costs <- get_costs(res_zeros)
    res_base_costs <- get_costs(res_base)

    # check costs for with drug scenario exactly 0
    expect_equal(as.numeric(res_zeros_costs[[2]]), 0)

    res_zero_icer <- get_icer(res_zeros)
    res_base_icer <- get_icer(res_base)

    # check ICER reduced
    expect_lt(res_zero_icer, res_base_icer)
  })
```

7.5 Population Tests

The transition matrix logic can be validated by forcing extreme mortality risks to modify life expectancy. This is a type of boundary value test. From Table 1, population focused tests correspond to T3, T4, T5. A demonstration of T4 is performed with the test case to set probability of death to 1 (i.e. 100%) immediately for everyone. The life expectancy is expected to be exactly 1 cycle. The state transition probability matrix for the intervention scenarios $s = 1, 2$ is

$$P^s = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

In the case of time-dependent transition probabilities, because the ‘Dead’ state is an absorbing state, simply setting $P^s(t = 1)$ would have the same effect. To implement the test case in the code, first take the shape of the transition probability matrix from `p_matrix` and then assign all probabilities to zero before assigning as one only the transition probabilities which go to the ‘Dead’ state.

```
test_that("Immediate mortality results in LE of 1", {
  # modify p_matrix to force death
  dead_p_matrix <- base_params$p_matrix
  dead_p_matrix <- 0
  dead_p_matrix[, "Dead", ] <- 1 # all transitions go to Dead

  res_death <- run_model(base_params, p_matrix = dead_p_matrix)

  # is Life Expectancy for both scenarios = 1 cycle
  expect_equal(as.numeric(get_le(res_death)), c(1, 1))
})
```

8 Automating the Process

In Wickham and Bryan, [21] they define three different scales at which test can be run. The case study example above can fit into the first, called *micro-iteration*, which is the interactive phase useful for refining and debugging. These tests do not have to live inside of an R package but can be run directly.

The true power of unit testing comes from automation. The entire test suite can be run all at once to ensure that new changes to the model have not broken existing logic, a so-called *macro-iteration* scale of testing [21].

Practically, all tests in the package can be run with `devtools::test()`. The typical output will be a report in the console with testing details and a summary similar to the following.

```
[ FAIL 0 | WARN 0 | SKIP 0 | PASS 3 ]
```

Where **PASS** means the model logic is holding up. **FAIL** means something is broken. The error message will say exactly which logic check failed (e.g., “Discounted QALYs were not less than undiscounted QALYs”). Any failed test identified can then be investigated and fixed so that it passes.

9 AI-Assisted Testing for Health Economic Models

The emergence of large language model (LLM) assistants such as Claude¹ represents a practical opportunity to accelerate and broaden the adoption of automated testing in health economic modelling. Rather than replacing the modeller’s domain judgement, AI can reduce the authoring burden for predefined tests and boilerplate test code, and lower the entry barrier for analysts who are less comfortable with software engineering conventions.

9.1 Generating *testthat* Scaffolding

A major practical obstacle to writing tests is the time required to translate a conceptual check such as in Table 1 into syntactically correct `testthat` code. An LLM can help perform this translation almost instantaneously when given a clear description of the model’s architecture and the intended check.

¹<https://www.anthropic.com/claude>

An LLM will produce code structurally equivalent to the `test_that()` block presented in Section 7, complete with informative labels and appropriate `expect_lt()` assertions. The modeller’s remaining task is to verify correctness, not to write from scratch. This is analogous to a compiler warning: the tool catches the mechanical, while the expert retains responsibility for the semantically meaningful.

9.1.1 Standardised Context and Prompts

To be effective, these agents require a project-level context anchor. This is a structured configuration file (e.g., `model-context.md`) that explicitly defines the project’s persona, testing standards, and domain-specific vocabulary. By providing these metadata, modellers can guide agents to generate tests that align with the HTA protocols (e.g., ZIN guidelines) and the specific modular architecture of their model. This ensures the agent does not simply suggest generic code.

The agreed health economic modelling tests can be included in the context as a source of truth. Table 1 is provided in the form of a test *registry* approach. The `model-context.md` file acts as a bridge between the HTA Registry (the requirements) and the R implementation (the syntax). With this file in the project root, we can simply provide a prompt to the AI agent, such as: “Generate the `testthat` code for T06 and T12 based on the provided `model-context.md`.” The agent will now automatically know which `run_model()` arguments to use, which getter functions are available, and how to label the test for auditability. An example `model-context.md` is provided in the Appendix.

9.2 Limitations of AI-Assisted Testing

AI assistance introduces its own failure modes that modellers must remain vigilant about. First, LLMs can generate tests that are syntactically correct but semantically wrong, for example, asserting that discounted costs are *greater* than undiscounted costs, which inverts the intended logic. Second, over-reliance on AI-generated tests may create a false sense of completeness: an LLM cannot know which clinical assumptions are model-specific and which are universal. Third, as with any generated code, tests must be read and understood by the modeller before being committed; committing unreviewed tests undermines the very accountability they are intended to provide. In summary, AI is best thought of as an efficient first-draft author, not as a subject-matter expert.

10 Discussion

The integration of automated testing into health economic modelling requires a non-trivial upfront investment in time and computational resources. Modellers must dedicate initial effort to operationalizing clinical and economic concepts into formal, machine-readable expectations. However, this initial expenditure yields a substantial return on investment (ROI) over the model’s lifecycle. Automated test suites drastically reduce the time spent debugging complex errors, as logic failures are caught immediately at the unit level rather than propagating silently into final ICER calculations. Furthermore, a comprehensive test suite facilitates seamless model updates and adaptations; modellers can confidently add new health states or treatments, knowing that automated checks will immediately flag any unintended consequences to existing code. Ultimately, adopting these rigorous software

engineering practices can significantly accelerate peer and agency review processes, as HTA bodies are presented with a transparent, computationally verified framework.

Despite its clear advantages, the widespread adoption of automated testing in health economics faces several practical limitations. The first is the steep learning curve for analysts who do not have formal backgrounds in software engineering. Transitioning from visually intuitive spreadsheet environments to abstract, script-based testing frameworks requires a fundamental shift in technical competencies.

In addition, it is crucial to recognize the conceptual boundaries of the tests described in this tutorial. Passing an automated test suite guarantees strict code verification (internal consistency), but does not equate to clinical validation (external validity). As outlined by the ISPOR-SMDM Modelling Good Research Practices Task Force, verifying the accuracy of mathematical coding is an essential prerequisite, but it must be accompanied by broader validation efforts—such as face validity, cross-validity, and the comparison of simulated results against real-world epidemiological data (Eddy et al., 2012). Automated testing confirms that the model is built right, but external clinical validation is still required to ensure that the right model was built.

The implementation of standard unit testing serves as a gateway to highly advanced quality assurance workflows. A natural next step is the adoption of Continuous Integration (CI) and Continuous Deployment (CD) pipelines hosted on version-control platforms like GitHub. CI frameworks automatically execute the entire *testthat* suite every time a code modification is pushed to the repository, entirely preventing broken code from merging into the main project.

A promising means of lowering this learning curve is the use of LLM assistants, such as Claude or similar tools, to accelerate the authoring of *testthat* code. Given a plain-language description of an intended check and the relevant function signatures, an LLM can produce a syntactically correct test block in seconds, effectively acting as a first-draft author that the domain expert then reviews and validates. This workflow is described in detail in Section 9. Importantly, however, AI assistance does not remove the need for expert judgement: a generated test that inverts the intended inequality (e.g., asserting discounted QALYs are *greater* than undiscounted QALYs) is syntactically indistinguishable from a correct one and will pass silently unless the modeller reads and understands what has been committed. AI tools are therefore best positioned as efficiency aids within a human-led review process, not as autonomous validators.

Looking ahead, the health economics community would benefit from establishing a universal, standardized suite of automated verification tests, ideally codified and endorsed by professional bodies such as ISPOR or SMDM. Having a centralized, open-source library of baseline HTA tests would unify global validation standards.

Finally, as computational literacy within the field matures, health economic modelling should follow a full software development process. Model building should begin with structured requirements capture and formal documentation. This could allow them to embrace Test-Driven Development (TDD). [2] In a TDD paradigm, modellers write unit tests based on the requirements of HTA protocols before writing the underlying model functions. This approach inherently forces modellers to define exact clinical expectations and boundary conditions at the inception of the project, ensuring that the model’s architecture is guided by rigorous validation criteria from day one.

Modellers can also expand their testing toolkits by using packages like *checkmate*, which provides fast, optimized assertions and custom *testthat*-based unit tests designed for complex R objects [11]. Furthermore, tests can be packaged up for ease of future use,

and tools such as the *covr* package can be implemented to formally evaluate overall test coverage.

Future research should focus on developing a comprehensive, standardized suite of verification tests. Ideally, this effort could be formalized and endorsed by professional organizations such as the International Society for Pharmacoeconomics and Outcomes Research (ISPOR) or the Society for Medical Decision Making (SMDM) to unify global validation standards.

Ultimately, as computational literacy within the field matures and health economic modelling follows a full software development process, the safety-critical compliance frameworks seen in other industries—such as A-SPICE and FMVSS—can act as a guidance and roadmap for standardizing integration testing in HTA.

11 Conclusion

As health economic models grow in structural and computational complexity, and the field continues its paradigm shift toward open-source languages like R, there is a critical need to adopt foundational quality-assurance techniques from software engineering. A demonstration of how integrating automated testing, specifically utilizing the *testthat* package, provides a rigorous, hands-on framework for internal model verification has been given. By systematically validating mathematical logic, transition dynamics, and extreme-value boundaries at the component level, modellers can proactively identify and resolve silent computational errors before they propagate through the model. Ultimately, embracing these testing workflows is essential to ensure computational accuracy, improve transparency, and prevent consequential errors in final health technology assessment and reimbursement decisions.

Code Availability Statement

All R code, test scripts, and the accompanying case study generated for this tutorial are publicly available through an open-source GitHub repository at [\[GitHubLink\]](#) to facilitate immediate adoption and replication.

References

- [1] F. Alarid-Escudero et al. A need for change! A coding framework for improving transparency in decision modeling. *PharmacoEconomics*, 37(11):1329–1339, 2019.
- [2] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley Professional, 2002.
- [3] J. Jaime Caro, Andrew H. Briggs, Uwe Siebert, and Karen M. Kuntz. Modeling good research practices—overview: A report of the ISPOR-SMDM modeling good research practices task force-1. *Value in Health*, 15(6):796–803, 2012.
- [4] E. J. Dasbach and E. H. Elbasha. Verification of decision-analytic models for health economic evaluations: An overview. *PharmacoEconomics*, 35:673–683, 2017.

- [5] David M. Eddy, William Hollingworth, J. Jaime Caro, Joel Tsevat, Kathryn M. McDonald, and John B. Wong. Model transparency and validation: A report of the ISPOR-SMDM modeling good research practices task force-7. *Value in Health*, 15(6):843–850, 2012.
- [6] Nathan Green, Felicity Lamrock, Nichola Naylor, Jack Williams, and Andrew Briggs. Health economic evaluation using markov models in R for microsoft excel users: A tutorial. *PharmacoEconomics*, 41:5–19, 2022.
- [7] Ross Hart, David Burns, Bram Ramaekers, Shijie Ren, Dan Gladwell, William Sullivan, Nicola Davison, Owen Saunders, Ian Sly, Tom Cain, and Dalia Lee. R and Shiny for cost-effectiveness analyses: why and when? a hypothetical case study. *PharmacoEconomics*, 38(8):765–776, 2020.
- [8] Jim Hester, Lionel Henry, Kirill Müller, and Thomas Lin Pedersen. *withr: Run Code ‘With’ Temporarily Modified Global State*, 2024. R package version 3.0.0.
- [9] D. Husereau et al. Consolidated health economic evaluation reporting standards (CHEERS) statement. *European Journal of Health Economics*, 14(3):367–372, 2013.
- [10] Hawre Jalal, Petros Pechlivanoglou, Eline Krijkamp, Fernando Alarid-Escudero, Eva Enns, and M G Myriam Hunink. An overview of R in health decision sciences. *Medical Decision Making*, 37(6):735–746, 2017.
- [11] Brent Laster. *Learning GitHub Actions: Automation and Integration of CI/CD with GitHub*. O’Reilly Media, 2023.
- [12] C. McCabe and S. Dixon. Testing the validity of cost-effectiveness models. *PharmacoEconomics*, 17(5):501–513, 2000.
- [13] National Highway Traffic Safety Administration. Federal motor vehicle safety standards. 49 C.F.R. § 571, 2026. U.S. Department of Transportation.
- [14] Nichola R. Naylor, Jack Williams, Nathan Green, Felicity Lamrock, and Andrew Briggs. Extensions of health economic evaluations in R for microsoft excel users: A tutorial for incorporating heterogeneity and conducting value of information analyses. *PharmacoEconomics*, 41:21–32, 2022.
- [15] Mark Roberts, Louise B. Russell, A. David Paltiel, Michael Chambers, Phil McEwan, and Murray Krahn. Conceptualizing a model: A report of the ISPOR-SMDM modeling good research practices task force-2. *Value in Health*, 15(6):804–811, 2012.
- [16] R. Smith, W. Mohammed, and P. Schneider. Packaging cost-effectiveness models in R: A tutorial. *Wellcome Open Research*, 8:419, 2023.
- [17] R. A. Smith, Y. Samyshkin, W. Mohammed, et al. assertHE: an R package to improve quality assurance of HTA models. *Wellcome Open Research*, 9:701, 2024.
- [18] P. Tappenden and J. B. Chilcott. Avoiding and identifying errors and other threats to the credibility of health economic models. *PharmacoEconomics*, 32:967–979, 2014.

- [19] VDA QMC. *Automotive SPICE Process Reference Model / Process Assessment Model v4.0*. Verband der Automobilindustrie (VDA), 2023. Available at <https://www.vda-qmc.de/>.
- [20] Hadley Wickham. testthat: Get started with testing. *The R Journal*, 3:5–10, 2011.
- [21] Hadley Wickham and Jennifer Bryan. *R Packages: Organize, Test, Document, and Share Your Code*. O’Reilly Media, 2nd edition, 2023.
- [22] Zorginstituut Nederland. Richtlijn voor het bouwen van kosteneffectiviteitsmodellen in R [guideline for building cost-effectiveness models in R]. Guideline, Zorginstituut Nederland, Diemen, The Netherlands, 12 2022.

A Appendix

A.1 Custom Expectations

Custom helper functions are highly effective tools for encapsulating repetitive, model-specific testing logic into clean, readable commands. There is a more formal way of creating custom expectations with *testthat* but for this purposes simple wrapper functions can be created. [20]

```
data(test_data)

test_that("QALY calculations", {

  # T01: QALYs are 0 if discount rate is 1 (all future value is 0)
  check_model_qalys(
    data = test_data, discount_rate = 1,
    expected_qalys = 0,
    label = "T01: QALYs with discount_rate = 1 should be 0")

  # T04: QALYs are 0 if all utilities are 0
  check_model_qalys(
    data = test_data, u_healthy = 0, u_sick = 0,
    expected_qalys = 0,
    label = "T04: QALYs with all utilities = 0 should be 0")

  # T03: QALYs with standard discount < undiscounted QALYs
  compare_model_runs(
    extractor_fn = get_qalys,
    comparison_fn = expect_lt,
    params_1 = list(discount_rate = 0.035),
    params_2 = list(discount_rate = 0),
    data = test_data,
    label = "T03: Discounted should be less than undiscounted QALYs"
  )

  # T02: Undiscounted QALYs (with u=1) should equal LE
  # This is a dynamic check
```

```

# (comparing two *different* outputs from the *same* run),
# so it doesn't fit the helper patterns
# and is clearer to write manually.
test_run_t02 <- run_model(test_data, discount_rate = 0,
                          u_healthy = 1, u_sick = 1)

expect_equal(
  get_qalys(test_run_t02), get_le(test_run_t02),
  label = "T02: Undiscounted QALYs (with u=1) should equal LE")
})

test_that("Cost calculations", {

  # T05: Costs with standard discount < undiscounted costs
  compare_model_runs(
    extractor_fn = get_costs,
    comparison_fn = expect_lt,
    params_1 = list(discount_rate = 0.035),
    params_2 = list(discount_rate = 0),
    data = test_data,
    label = "T05: Discounted should be less than undiscounted costs"
  )

  # T06: If all costs are 0, total cost should be 0
  check_model_costs(
    data = test_data,
    c_healthy = 0,
    c_sick = 0,
    c_intervention = 0,
    c_death = 0,
    expected_costs = 0,
    label = "T06: Total cost should be 0 when all cost inputs are 0"
  )

  # T07: Full discount (rate = 1) < standard discount
  # (Note: Assumes non-zero costs after cycle 0)
  compare_model_runs(
    extractor_fn = get_costs,
    comparison_fn = expect_lt,
    params_1 = list(discount_rate = 1),
    params_2 = list(discount_rate = 0.035),
    data = test_data,
    label =
      "T07: Full discount Costs should be less than standard discount"
  )
})

test_that("Life Expectancy (LE) calculations", {

  # T08: LE should not be affected by discount rate
  compare_model_runs(
    extractor_fn = get_le,

```

```

    comparison_fn = expect_equal,
    params_1 = list(discount_rate = 0),
    params_2 = list(discount_rate = 0.05),
    data = test_data,
    label = "T08: LE should not be affected by discount rate"
  )

# T09: If all transition probabilities to death are 1, LE is 1 cycle
# (Assuming model starts in cycle 1 and everyone lives 1 cycle)
check_model_le(
  data = test_data,
  p_healthy_death = 1,
  p_sick_death = 1,
  expected_le = 1,
  label = "T09: LE should be 1 when all p_death = 1"
)

# T10: If all transition probabilities to death are 0, LE is n_cycles
n_cycles_test <- 50
check_model_le(
  data = test_data,
  p_healthy_death = 0,
  p_sick_death = 0,
  n_cycles = n_cycles_test,
  expected_le = n_cycles_test,
  label = "T10: LE should be n_cycles when all p_death = 0"
)
})

```

The custom expectation functions used in the previous tests are defined as follows.

```

check_model_qalys <- function(test_data, discount_rate, expected_qalys) {
  # test_run <-
  #   test_data |>
  #   set_discount_rate(discount_rate) |>
  #   run_model()
  # # do.call(what = run_model, args = _)

  test_run <- run_model(test_data, discount_rate = discount_rate)
  expect_equal(get_qalys(test_run), expected_qalys)
}

#' Run model and check QALYs against an expected value
#'
#' @param expected_qalys The exact expected QALY value.
#' @param ... Arguments passed to 'run_model()'.
#' @param label A label for the test, passed to 'expect_equal()'.
check_model_qalys <- function(expected_qalys, ..., label = NULL) {
  run <- run_model(...)
  expect_equal(get_qalys(run), expected_qalys, label = label)
}

```

```

#' Run model and check Costs against an expected value
#'
#' @param expected_costs The exact expected Cost value.
#' @param ... Arguments passed to 'run_model()'.
#' @param label A label for the test, passed to 'expect_equal()'.
check_model_costs <- function(expected_costs, ..., label = NULL) {
  run <- run_model(...)
  expect_equal(get_costs(run), expected_costs, label = label)
}

#' Run model and check Life Expectancy against an expected value
#'
#' @param expected_le The exact expected LE value.
#' @param ... Arguments passed to 'run_model()'.
#' @param label A label for the test, passed to 'expect_equal()'.
check_model_le <- function(expected_le, ..., label = NULL) {
  run <- run_model(...)
  expect_equal(get_le(run), expected_le, label = label)
}

#' Compare model outputs from two different parameter sets
#'
#' @param extractor_fn Function to get the result (e.g., get_qalys).
#' @param comparison_fn testthat expectation function.
#' @param params_1 List of parameters for first 'run_model()' call.
#' @param params_2 List of parameters for second 'run_model()' call.
#' @param data Base dataset (passed to 'run_model()').
#' @param label Label for test, passed to comparison function.
compare_model_runs <- function(extractor_fn, comparison_fn,
                               params_1, params_2, data, label = NULL) {

  run_1_args <- c(list(data = data), params_1)
  run_1 <- do.call(run_model, run_1_args)
  output_1 <- extractor_fn(run_1)

  run_2_args <- c(list(data = data), params_2)
  run_2 <- do.call(run_model, run_2_args)
  output_2 <- extractor_fn(run_2)

  comparison_fn(output_1, output_2, label = label)
}

```

A.2 Advanced Testing Approaches

A.2.1 Reproducible Stochastic Tests with withr

Probabilistic Sensitivity Analysis involves random sampling, so stochastic tests must set a seed to be reproducible. The `withr` package [8] provides `with_seed()`, which temporarily sets the RNG seed for the duration of a single expression and then restores the previous state. This avoids the side-effect of a `set.seed()` call persisting into subsequent tests and affecting their behaviour.

```

test_that("PSA beta samples remain within (0, 1)", {
  withr::with_seed(42, {
    psa_samples <- sample_psa_parameters(
      n_sim      = 1000,
      base_params = base_params
    )
    beta_cols <- grep("^utility_", names(psa_samples), value = TRUE)
    for (col in beta_cols) {
      expect_true(
        all(psa_samples[[col]] > 0 & psa_samples[[col]] < 1),
        label = paste0("T13: beta-sampled '", col,
                        "' must lie strictly in (0, 1)")
      )
    }
  })
})

```

A.2.2 Parameterised Tests with purrr

Tests T01 and T08 in Table 1 share an identical structure: run the model with an extreme discount rate and assert that the corresponding output approaches zero. Duplicating `test_that()` blocks for each output type (QALYs, costs) increases maintenance burden. The `purrr` package [21] enables parameterised iteration over a list of such cases, keeping the test file concise and the intent explicit.

```

# Define parameter sets for "extreme discount -> output near zero" tests
extreme_discount_cases <- list(
  list(label      = "T01/T08: QALYs approach 0 at extreme discount",
        getter     = get_qalys,
        index      = "without_drug"),
  list(label      = "T01/T08: Costs approach 0 at extreme discount",
        getter     = get_costs,
        index      = "without_drug")
)

purrr::walk(extreme_discount_cases, function(case) {
  test_that(case$label, {
    res <- run_model(base_params, discount_rate = 1000)
    expect_lt(
      case$getter(res)[[case$index]], 1,
      label = case$label
    )
    expect_equal(
      case$getter(res)[[case$index]], 0,
      tolerance = 0.01,
      label     = case$label
    )
  })
})

```

A.2.3 Richer Failure Messages

By default, a failing `expect_equal()` reports the actual and expected values but not which model scenario or cycle produced the discrepancy. Passing a descriptive string to the `label` argument, and, where useful, a longer diagnostic string to the `info` argument greatly accelerates root-cause analysis, particularly when many tests are run in a CI pipeline and the failure must be diagnosed without an interactive R session.

```
test_that("Immediate mortality results in LE of 1 cycle", {
  dead_p_matrix      <- base_params$p_matrix
  dead_p_matrix[]    <- 0
  dead_p_matrix[, "Dead", ] <- 1

  res_death <- run_model(base_params, p_matrix = dead_p_matrix)
  le_values <- as.numeric(get_le(res_death))

  expect_equal(
    le_values,
    c(1, 1),
    label = "T05: LE must equal 1 cycle when all p_death = 1",
    info  = paste("Observed LE values:",
                  paste(round(le_values, 4), collapse = ", "))
  )
})
```

A.3 AI Agent Project Context

```
# AI Agent Project Context: Health Economic Model Verification
```

```
## 1. Persona & Scope
```

```
You are an expert Health Economist and R Software
Developer. Your goal is to write robust, transparent,
and reproducible unit tests for HTA models using the
'testthat' package.
```

```
## 2. Structural Contract: Helper Functions
```

```
To ensure modularity and "separation of concerns," all
model interactions must follow these interface patterns:
```

```
* **Required Setters/Mutators** (Must return input object):
```

```
* 'set_discount_rate(input, discount_rate)'
* 'set_costs(input, scenario, values)'
* 'set_p_matrix(input, scenario, values)'
* 'set_utilities(input, scenario, values)'
* 'set_time_horizon(input, scenario, values)'
```

```
* **Required Getters/Accessors**:
```

```
* 'get_qalys(res)': Extract total QALYs.
* 'get_costs(res)': Extract total costs.
* 'get_le(res)': Extract Life Expectancy.
* 'get_icer(res)': Extract ICER.
```

```
* **Required Wrappers & Utilities**:
```

- * 'run_model(data, ...)': Mandatory wrapper.
- * 'compare_model_runs(...)': Encapsulate comparison logic for different model outputs.

3. Test Registry (Source of Truth)

All automated verification must align with these:

ID	Group	Test Condition	Expected Result
T01	QALY	Set discount rate = 1	All future QALYs = 0
T02	QALY	Set discount rate = 0 (u=1)	Total QALYs = LE
T03	Pop	Set relative effects = 1	Equal LYGs & QALYs
T04	Pop	Set mortality = 0	LE = time horizon
T05	Pop	Set mortality = 1	LE = 1
T06	QALY	Set utilities = 1	QALY gains = LYGS
T07	QALY	Positive discount rate	Disc QALYs < Undisc
T08	QALY	QALY discount rate = inf	QALYs t>0 -> 0
T09	Cost	Set intervention costs = 0	ICER is reduced
T10	Cost	Increase intervention costs	ICER is increased
T11	Cost	Set cost discount rate = 0	Disc costs = Undisc
T12	Cost	Set cost discount rate = inf	Costs t>0 -> 0
T13	Input	Sampled parameters	Bounds (e.g. 0-1)

4. Coding Standard & Implementation Rules

- * **Modular Architecture**: "Separation of concerns" is mandatory. Do not write monolithic scripts.
- * **TDD**: Define expectations based on requirements before writing model functions.
- * **Naming**: Every 'test_that()' block must start with its Registry ID.
- * **Tolerance**: Use 'tolerance = 0.01' for floating-point comparisons.